

GitHub Copilot for Enterprise Teams — Participant Lab Guide

Synechron Technologies Pvt. Ltd. | 1-Day Training

This guide is yours to keep. Every exercise, every prompt, every reference for the day is here. The morning is mostly demo-driven — you watch. The afternoon is hands-on — you build.

Contents

- [Before You Arrive](#)
- [Lab 1.1 — Prove the Mental Model](#)
- [Lab 2.1 — Rewrite Three Weak Prompts](#)
- [Lab 2.2 — Try the Techniques](#)
- [Lab 3.1 — Your First `copilot-instructions.md`](#)
- [Lab 3.2 — Comment-Driven Development](#)
- [Lab 4.1 — Built-in Slash Commands with Direction](#)
- [Lab 4.2 — Build Your Own Slash Command](#)
- [Lab 4.3 — Agent Mode for a Multi-File Refactor](#)
- [Lab 6 — Build Sprint](#)
- [Quick Reference Card](#)

Before You Arrive

Software

Tool	Why	Install
Git	Version control	https://git-scm.com/downloads
Node.js (LTS)	Runtime for the build sprint	https://nodejs.org
Visual Studio Code	Editor (latest stable)	https://code.visualstudio.com
GitHub CLI	gh copilot, auth	https://cli.github.com

GitHub

- Active GitHub account.
- GitHub Copilot **paid seat** assigned and active. Verify at <https://github.com/settings/copilot>. You should see your subscription confirmed.
- GitHub CLI authenticated: run `gh auth login` and follow the browser flow.

VS Code Extensions

Install these from the Extensions panel in VS Code:

- **GitHub Copilot**
- **GitHub Copilot Chat**
- **REST Client** (Huachao Mao)
- **ESLint**

- **Prettier**

Verify You're Ready

In a terminal:

```
git --version
node --version
code --version
gh --version
gh auth status
```

All five should succeed. If any fail, contact the trainer before the session.

In VS Code:

- Open an empty `.ts` file.
- Type `// fibonacci function` and press Enter.
- You should see grey "ghost text" appear with a suggestion. Press Tab to accept.

If you don't see ghost text — your Copilot extension isn't connected. Check the Copilot icon in the bottom status bar.

Lab 1.1 — Prove the Mental Model

Time: 3 min (during Module 1) **Goal:** See for yourself that the same prompt can produce different output, and that specificity dramatically improves consistency.

Steps

1. Open VS Code. Create a new file `parser.ts`.
2. Type this comment and press Enter:

```
// parse a date
```

Note what Copilot suggests. Reject (`Escape`).

3. Delete and re-type the same comment. Note the suggestion. Different? Reject.
4. Replace with this:

```
// Parse an ISO-8601 date string and return a Date object.
// Return null if the input is invalid. Do not throw.
```

Press Enter. Compare to attempts 1 and 2.

What to notice: the third version is more consistent across runs, and almost certainly more correct. You didn't change the model. You changed the prompt.

Lab 2.1 — Rewrite Three Weak Prompts

Time: 15 min (Module 2) **Goal:** Apply the Prompt Formula on weak prompts and see the output quality jump.

The Formula

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT

Weak Prompts

For each, write a strong version using the formula, then run both in Copilot Chat. Note the difference.

Prompt 1: add caching to this function

Set up: paste this function into a new `cache-target.ts` file. Select it.

```
export async function getUserProfile(userId: string) {
  const response = await fetch(`https://api.example.com/users/${userId}`);
  if (!response.ok) {
    throw new Error(`Failed to fetch user ${userId}`);
  }
  return response.json();
}
```

Run weak prompt as Inline Chat (`Ctrl+I` / `Cmd+I`):

add caching to this function

Now write your strong version. Hints — what does Copilot need to know?

- What kind of cache? In-memory? Redis? Browser?
- TTL?
- Cache key strategy?
- Invalidation?
- What library is allowed?
- What's the cache hit/miss behaviour for errors?

Your strong version (write here):

[your prompt]

Run it. Compare the two outputs.

Prompt 2: write tests

Set up: keep the same `cache-target.ts` file. Select the function.

Weak prompt:

write tests

Write your strong version. Hints:

- Test framework? (Vitest, Jest, Mocha, ...)
- Style? (`describe/it` , `test(...)` , etc.)
- Test approach for the `fetch` call — mock it? Use MSW? Use a fake server?
- Edge cases to cover?
- Test file naming convention?

- Where should it live?

Your strong version:

[your prompt]

Prompt 3: fix the bug

Set up: paste this deliberately buggy function:

```
export function findLastEven(numbers: number[]): number | undefined {
  for (let i = 0; i < numbers.length; i++) {
    if (numbers[i] % 2 === 0) {
      return numbers[i];
    }
  }
  return undefined;
}
```

(Bug: it returns the *first* even number, not the *last*.)

Weak prompt:

fix the bug

Write your strong version. Hints:

- What is the expected behaviour?
- How do you know there's a bug?
- What input should reproduce it?
- Constraints on the fix? (Must still be O(n)? Must not mutate input?)

Your strong version:

[your prompt]

Debrief Questions

- Which weak prompt was most damaging — i.e. produced the worst-quality first attempt?
- For your strong versions, which element of the formula made the biggest difference: context, constraints, or output format?

Lab 2.2 — Try the Techniques

Time: Self-paced; explore during breaks or after the session. **Goal:** Build hands-on familiarity with each prompting technique.

For each technique below, try the example in Copilot Chat and observe what Copilot does.

Zero-Shot

Write a TypeScript function `slugify(input: string): string` that converts a string to a URL-safe slug. Replace spaces and underscores with hyphens,

lowercase, strip non-alphanumeric except hyphens, collapse multiple hyphens.

Few-Shot

Convert these snake_case strings to camelCase. Examples:

snake_case_string → snakeCaseString
api_response_data → apiResponseData
user_id → userId

Now convert: shipping_address_line_2

Chain of Thought

Before writing the code, list the edge cases for parsing a CSV row that may contain quoted fields with embedded commas, escaped quotes, and trailing whitespace. Then write the parser function in TypeScript handling each case.

Role Prompting

Acting as a senior application security engineer reviewing code for OWASP Top 10 issues, review the following Express route handler and identify any security concerns:

[paste a route handler]

Micro-Task

Don't ask "build the user module." Break it down:

Write only the Zod schema for the user creation request body. Fields: email (valid email), password (min 8 chars, at least one digit), name (1-100 chars). Output only the schema, nothing else.

Then a separate prompt:

Write only the controller function that uses the schema above to validate and call userService.create(). Return 201 with the new user on success, 400 with structured error on validation failure.

Stepwise

Implement a CSV export endpoint in three steps:

Step 1: Define the TypeScript type for the data row.

Step 2: Write the function that converts an array of those rows to a CSV string, handling quoting and escaping.

Step 3: Write the Express route handler that calls the converter and sets the correct Content-Type and Content-Disposition headers for download.

Do them in order.

Specification-First

```
/**
 * SPEC:
 *
 * Function: rateLimit
 * Purpose: Token-bucket rate limiter middleware for Express.
 *
 * Configuration:
 *   - maxRequests: number of requests allowed
 *   - windowMs: time window in milliseconds
 *
 * Behaviour:
 *   - Track per-IP via req.ip.
 *   - Return 429 with Retry-After header when exceeded.
 *   - Reset bucket on a sliding window basis.
 *
 * Constraints:
 *   - No external dependencies.
 *   - Memory only (single-instance, no Redis).
 *   - Type-safe with TypeScript strict mode.
 */
// Implement based on the spec above.
```

Cursor-Aware

Place your cursor inside an existing function body where you want a suggestion. Copilot reads forward and backward — that placement matters.

```
function processOrder(order: Order) {
  validateOrder(order);
  // <- place cursor here, type "// charge the customer", press Enter
  notifyWarehouse(order);
}
```

Context Injection via Comments

```
// Note: the `users` table uses `id` (UUID), `email`, `created_at`.
// It has NO `name` column – names are in the separate `profiles` table.
// Query users joined with their profile name.
```

Output Scaffolding

```
export class OrderService {
  constructor(
```

```

    private readonly orderRepository: OrderRepository,
    private readonly paymentGateway: PaymentGateway,
    private readonly logger: Logger,
  ) {}

  async placeOrder(input: PlaceOrderInput): Promise<Order> {
    // 1. Validate input

    // 2. Reserve inventory

    // 3. Charge payment

    // 4. Persist the order

    // 5. Emit order.placed event

    // 6. Return the order
  }
}

```

Constraint-Based

Write a function to deeply clone a JavaScript object.

CONSTRAINTS:

- No external libraries (no lodash).
- No `structuredClone` (must work in older Node).
- Must handle: nested objects, arrays, Dates, Maps, Sets.
- Must NOT handle: functions, class instances (return as-is).
- TypeScript with full type preservation via generics.

Positive + Negative Framing

Refactor this function to be more readable.

PREFER:

- Early returns over nested if/else.
- Named constants over magic numbers.
- Pure functions where possible.

AVOID:

- Mutation of input arguments.
- Renaming public function/variable names that other files depend on.

Iterative Refinement

Round 1:

Write a function to merge two sorted arrays.

Round 2 (on the same chat):

Now make it generic – accept a comparator function so it works for any type, not just numbers.

Round 3:

Now add JSDoc with @param, @returns, and an @example block.

Prompt Chaining

Round 1:

Define the TypeScript interface for an OrderEvent. Fields: orderId (UUID), status (one of: placed, paid, shipped, delivered, cancelled), occurredAt (ISO timestamp), payload (record with arbitrary key/values).

Round 2 (use Round 1's output as the foundation):

Using the OrderEvent interface from above, write a discriminated union type EventPayload so each status has its strongly-typed payload shape.

Lab 3.1 — Your First copilot-instructions.md

Time: Self-paced during Module 3. **Goal:** Write a starter instructions file for your team's standards.

In the Build Sprint repo (or any of your own repos), create `.github/copilot-instructions.md`. Fill in the sections below with **your team's** actual standards.

```
# Project Coding Standards for GitHub Copilot

## Language & Framework
- [What language and version? TypeScript strict? Node version? Framework?]

## Architecture
- [Layered? Hexagonal? MVC? Where does business logic live?]
- [Folder structure conventions?]

## Validation & Errors
- [Validation library?]
- [Error response shape?]
- [Async style?]

## Naming
- [Variables / functions / types / files?]

## Testing
- [Framework? Style? Where do tests live?]

## Logging
- [Library? Levels? What never to log?]
```


What NOT to do

- [Common anti-patterns in your codebase that should not appear in new code.]

Save it. Commit it. Watch your team's Copilot output improve over the next week.

Lab 3.2 — Comment-Driven Development

Time: 5 min during Module 3. **Goal:** Feel the difference between a one-line comment and a spec block.

Create `price-calculator.ts`. Try each style in order.

Style A — One line

```
// Calculate VAT
```

Press Enter. What does Copilot suggest?

Style B — Spec block

```
/**
 * Calculate VAT (Value Added Tax) for a given amount.
 *
 * @param amount - Pre-tax amount in paise (smallest currency unit).
 * @param vatRate - VAT rate as decimal (e.g., 0.18 for 18%).
 * @returns The VAT amount, rounded to the nearest paise.
 *
 * Edge cases:
 * - Negative amounts: throw RangeError.
 * - Zero amount: return 0.
 * - vatRate outside [0, 1]: throw RangeError.
 */
```

Press Enter.

Style C — Spec + scaffold

```
/** ...same spec as above... */
export function calculateVat(amount: number, vatRate: number): number {
  // Validate inputs

  // Calculate raw VAT

  // Round to nearest paise
}
```

At each `//` comment, let Copilot fill the block.

Observation: the spec + scaffold pattern gives you the most predictable, reviewable output.

Lab 4.1 — Built-in Slash Commands with Direction

Time: 10 min during Module 4. **Goal:** Stop using slash commands as fire-and-forget. Direct them.

Use any function from the Build Sprint (or paste one from the morning).

For each, try the bare version first, then the directed version. Compare.

/explain

Bare:

```
/explain
```

Directed:

```
/explain in 3 bullet points, focused only on the side effects and any I/O.  
Skip the obvious. Assume I'm a senior dev.
```

/fix

Bare:

```
/fix
```

Directed:

```
/fix the bug, but first explain in one sentence what the bug is, then show  
the corrected code. Do not refactor anything beyond the bug.
```

/tests

Bare:

```
/tests
```

Directed:

```
/tests using Vitest with describe/it style. Cover:  
- Happy path  
- Empty input  
- Single-element input  
- Input with one invalid item among many  
  
Do not add tests for input types that TypeScript already prevents at compile time.
```

/doc

Bare:

```
/doc
```

Directed:

```
/doc include @param, @returns, @throws, and an @example block. Keep the
summary line under 80 characters. Use British English.
```

Lab 4.2 — Build Your Own Slash Command

Time: 15 min (during Module 4) or self-paced. **Goal:** Author a prompt file that becomes a reusable slash command for your team.

Setup

In a workspace, create `.github/prompts/`. Inside, create a `.prompt.md` file. The file name (minus extension) becomes the slash command name. Example: `review-security.prompt.md` becomes `/review-security`.

Template

```
---
description: One-line description shown in the command picker.
mode: ask
---
```

Acting as [role].

[Your instructions to Copilot]

For each [unit of work]:

- [What to produce]
- [In what format]
- [Any constraints]

Selected code:

`${selection}`

Additional context: `${input}`

Variables Available

- `${selection}` — the code the user has selected when invoking the command.
- `${input}` — anything the user typed after the command, e.g. `/review-security pay particular attention to authentication`.
- You can also reference files: `#file:path/to/file.ts` in the prompt body.

Ideas to Build (Pick One)

1. `/synechron-jsdoc` — Add JSDoc following Synechron's documentation standard (you define it).
2. `/log-instrument` — Add `pino` logger calls at strategic points (function entry, error paths, important branches).
3. `/express-controller` — Generate a controller scaffold following the team's layered architecture.
4. `/oas-update` — Update the OpenAPI spec to match the selected route handler.
5. `/code-review` — Apply your team's code review checklist as a single command.

Build one. Commit `.github/prompts/your-command.prompt.md` to the repo. Test by selecting code and typing `/your-command` in Copilot Chat.

Lab 4.3 — Agent Mode for a Multi-File Refactor

Time: 10 min during Module 4. **Goal:** Use Agent mode for something basic chat can't do well — a coordinated, multi-file change.

Setup

In the build sprint repo (or any small repo you have), pick a refactor that touches 3+ files. Example: rename a service, extract a constant, replace a logger.

Prompt

This lab has two parts. **Try both** — the contrast is the lesson.

Part A — Agent alone:

In Copilot Chat, switch the mode dropdown to **Agent**.

Prompt example (adjust for your repo):

```
Refactor: rename the `UserService` class to `AccountService` everywhere
in the codebase. Update all imports, all type references, and the file
name. Update any tests. Update any documentation. Follow our coding
standards in .github/copilot-instructions.md.
```

Part B — Plan first, then Agent:

Open a fresh Chat session. Switch to **Plan** mode.

Run the same prompt. Notice Copilot now produces an **implementation plan** instead of edits — a summary, the list of files it will touch, and any clarifying questions. Review the plan. Answer any questions. Then click **Start Implementation** to hand off to Agent.

What to Do

- For Part A: approve each step one at a time. **Do not click "approve all"** — you're learning to supervise.
- For Part B: review the plan carefully *before* clicking Start Implementation. Did Copilot miss a file? Was there an assumption that needed correcting? Catching it in the plan is cheaper than catching it in committed diffs.
- For each diff Copilot proposes, scan it. Reject any change that's wrong.
- Run your tests after Agent mode finishes. Do they still pass?

Reflection

- What did Agent mode do well in Part A? What did it miss?
 - Did Plan mode in Part B surface anything before execution that you'd have caught only in diff review with Part A?
 - Which approach do you think you'll use for a real multi-file refactor on Monday?
-

Lab 6 — Build Sprint

Time: 60 minutes. **Goal:** Build a working Task Management REST API end-to-end, applying everything from the morning.

The full project specification is in `BUILD_SPRINT_PROJECT.md`. Read it before you start.

Phase 1: Project Setup (15 min)

Use Copilot Chat in Agent mode to scaffold the project. If you want to review Copilot's plan before any files are created, start in **Plan mode**, read and refine the plan, then click **Start Implementation** to hand off to Agent. Write a strong prompt — use the formula.

You should end Phase 1 with:

- `package.json`, `tsconfig.json`, `.gitignore`
- `src/index.ts` (Express bootstrap, `/health` endpoint)
- `.github/copilot-instructions.md` (your team's standards)
- Folder structure: `src/routes`, `src/controllers`, `src/services`, `src/repositories`, `src/middleware`, `src/schemas`
- `npm install` complete, `npm run dev` should start the server.

Phase 2: Build the API (25 min)

Implement the five endpoints from the spec.

Recommended order:

1. Define the `Task` type and the in-memory repository first.
2. Build the service layer with stub implementations.
3. Build the controller that delegates to the service.
4. Wire up the route.
5. Repeat for each endpoint, but reuse what you've built.

Use Inline Chat for surgical edits. Use Agent mode when adding a new endpoint (touch route + controller + service + schema in one go).

Phase 3: Wire and Verify (20 min)

- Add Zod schemas for request bodies.
- Add a global error-handling middleware.
- Smoke test using the `.http` file included in the spec — use the REST Client extension to run requests one by one.

Checkpoint

By the end of the build sprint, you should be able to:

```
curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Test task", "description": "A test", "status": "todo"}'

curl http://localhost:3000/tasks
```

And both should work.

Quick Reference Card

Keyboard Shortcuts

Action	macOS	Windows / Linux
Accept suggestion	Tab	Tab
Reject suggestion	Esc	Esc
Cycle suggestions	Option+] / Option+[	Alt+] / Alt+[
Open suggestions panel	Cmd+Enter	Ctrl+Enter
Inline Chat	Cmd+I	Ctrl+I
Open Copilot Chat	Ctrl+Cmd+I	Ctrl+Alt+I

The Prompt Formula

ROLE + TASK + CONTEXT + CONSTRAINTS + OUTPUT FORMAT

Built-in Slash Commands

Command	Use For
/explain	Understand selected code
/fix	Fix a bug in selected code
/tests	Generate tests for selected code
/doc	Add documentation to selected code
/help	List all available commands
/clear	Start a fresh chat

Chat Context Directives

Syntax	Purpose
#file:path/to/file.ts	Pin a specific file as context
#selection	Reference the current editor selection
@workspace	Search the whole workspace
@terminal	Reference recent terminal output
@github	Search GitHub (issues, PRs, repos)

Chat Modes

Mode	What It Does
------	--------------

Ask	Answers questions, suggests code, never edits files
Plan	Generates an implementation plan for your review — summary, file-by-file step breakdown, clarifying questions. No code changes until you hand off to Agent
Agent	Plans + edits + runs commands autonomously across multiple files, you approve each step

Edit mode (from earlier versions of Copilot) was folded into Agent mode. The three modes above are what you'll see in the mode dropdown today.

When to Use Which

Task	Tool
1-line completion	Just type, hit Tab
Refactor in one file	Inline Chat (Cmd+I)
Multi-turn explore	Copilot Chat (Ask mode)
Coordinated multi-file change	Copilot Chat — Plan mode → review → Agent mode
Repeated team workflow	Custom slash command
Domain knowledge across repos	Custom Copilot Extension
Terminal task	gh copilot suggest / Copilot CLI

When Copilot Goes Quiet — Quick Fixes

- 1. Click the Copilot status bar icon → check state.
- 2. Developer: Reload Window from the Command Palette.
- 3. GitHub Copilot: Sign Out then sign in again.
- 4. Check <https://www.githubstatus.com>.
- 5. If a specific file: check if it's in your org's exclusion list (Copilot icon will show "disabled for this file").

End of Participant Lab Guide.

Keep this file. Reference it Monday. The Prompt Formula on the back of your monitor for a week is the cheapest productivity intervention you can make.